项目设计方案文档

项目名称	基于 eBPF 的系统级多智能体监测框架与方法 ——CoreSight
团队名称	熔岩芝士烤冷面
指导教师	李晓露
完成日期	2026.6.28

目录

目录 2
1 概述 5
1.1 项目亮点 5
1.2 赛题分析 6
1.3 背景调研 7
1.3.1 现有方案概览 7
1.3.2 本方案的独特优势 8
2 架构 10
2.1 架构图 10
2.2 整体架构说明 10
2.3 模块职责一览 11
3 设计思路 13
3.1 核心设计决策 13
3.2 技术选型 14
4 实现描述 16
4.1 多层级数据捕获 16
4.1.1 功能描述 16
4.1.2 实现细节 16
4.1.3 核心代码片段 20
4.2 intra-agent 22
4.2.1 功能描述 22
4.2.2 实现细节 23
4.2.3 核心代码片段 24
4.3 inter-agent 27
4.3.1 功能描述 27
4.3.2 实现细节 29
4.3.3 核心代码片段 33
4.4 归因定位 39
4.4.1 功能描述 40

4.4.2 实现细节 40
4.4.3 核心代码片段 41
4.5 CoreSight-showcase 47
4.5.1 功能描述 47
4.5.2 实现细节 47
5 代码说明 49
5.1 项目目录结构 49
5.1.1 整体 49
5.1.2 intra_agent 50
5.1.3 inter_agent 50
5.2 关键类/函数说明 52
6 实验说明 57
6.1 实验环境 57
6.2 依赖安装 57
6.3 编译/构建说明 58
6.4 使用说明 58
6.4.1 快速开始 58
6.4.2 功能演示 59
6.4.3 参数说明 60
7 系统测试 62
7.1 功能测试 62
7.2 性能测试 63
8 研发过程记录 65
8.1 遇到的问题及解决方法 65
8.1.1 问题一 65
8.1.2 问题二 65
8.2 设计迭代记录 67
9 未来规划 68
9.1 待完成工作 68
9.2 优化方向 68
10 开源/AI 说明 70
10.1 第三方开源组件 70

10.2 参考的开源项目/论文 70
10.3 AI 辅助生成内容说明 71
11 附录 72
附录 A：变更日志 72

1 概述

1.1 项目亮点

非侵入式内核态监测：基于 eBPF 技术，无需修改内核源码或重编译目标智能体程序，即可在内核态动态挂载监测探针，实现对智能体运行时的实时观测，充分体现 eBPF 在安全性与可观测性方面的独特优势。且针对不同类型 Agent 采取相应措施，有效捕获多种类 Agent 明文信息。

跨层数据关联与语义贯通：贯通应用层交互（Prompt/Response）与内核层系统调用（进程管理、文件操作、网络通信）的数据链路，突破用户态监测工具难以跨越的“语义鸿沟”，实现上层行为动机与底层系统异常的因果匹配。

单智能体异常检测：针对单个 Agent 运行时的资源滥用与安全风险，通过 eBPF 捕获其底层系统调用行为（高频 API 调用、重复提示词请求、无意义文件读写），识别逻辑死循环与资源浪费；同时检测非预期的 Shell 命令行解释程序启动、系统敏感文件越权访问、超出工作区范围的异常文件删除等高危操作。

多智能体异常监测：覆盖父子层级关系（Parent-Child / Hierarchical）与平级对等关系（Peer-to-Peer / Joint）两种典型多智能体协作拓扑。在父子层级模式下，父智能体接收任务并拆解，子智能体只听从父智能体指令调用工具，系统通过 Clone Map 模式自动追踪父子进程链，识别子 Agent 产生的异常操作并回溯至父 Agent 的指令源头；在平级对等模式下，智能体之间互为 Peer，拥有平等的工具调用权与话语权，通过互相接力（Handoff）或自由辩论完成任务，系统在进程交织的环境中精准定位异常根因，区分协作正常行为与异常竞争。

多维异常检测能力：综合应用层交互内容与底层行为数据，识别逻辑死循环与资源浪费、敏感文件越权访问、非预期 Shell 启动、多智能体协作异常等多种典型异常模式，并输出含时间戳、PID/TID、操作对象等关键上下文的归因告警。

1.2 赛题分析

赛题要求构建一个基于 eBPF 的非侵入式多智能体异常监测系统，核心挑战可以归纳为三个层面：**数据采集**（如何在内核态高效捕获多维系统调用数据）、**语义贯通**（如何将底层行为事件与上层的 Agent 身份及交互意图关联）、**异常识别**（如何在多 Agent 并发场景下准确判定异常并定位根因）。

数据采集层面：赛题要求覆盖进程管理、文件系统、网络通信三类基础系统调用，进阶要求通过 uprobes 截获 HTTPS 加密流量中的 Prompt/Response 明文。eBPF 的 kprobes/tracepoints 可满足基础要求，但 uprobes Hook 加密库的稳定性与性能开销是需要权衡的问题。此外，内核态采集与分析之间存在明显的吞吐量落差：内核探针可产生大量高频事件，直接传递给分析系统会造成瓶颈，因此需要引入中间缓冲层（如 Redis）进行数据暂存与流控，同时保存超长上下文，提供给分析模块。

语义贯通层面：赛题评审中“功能实现与异常回溯”占比最高（40%），这要求系统不仅记录“哪个 PID 执行了什么系统调用”，还要回答“该 PID 属于哪个 Agent”以及“这个 Agent 正在做什么”的问题。在零侵入约束下，Agent 身份识别无法通过修改应用代码实现，只能依赖进程父子关系链（Clone Map）和二进制路径、命令行参数等启发式特征进行推断。对于多 Agent 协作场景，进程空间的交织使得身份识别和因果归因更加复杂。

异常识别层面：赛题列出了资源异常（逻辑死循环）和安全异常（Shell 启动、文件越权）两大类基础目标，进阶目标要求识别多智能体协作异常。结合 UC 伯克利 MAST 分类法的研究成果，多智能体异常可进一步细分为规范违反、智能体间不一致、验证缺失三个子类。异常检测引擎需要同时处理单 Agent 的独立异常和跨 Agent 的协作异常，这要求架构上具备模块化的检测能力划分。

基于以上分析，我们设计了三层架构方案：

低层 eBPF 数据采集层：通过内核态探针捕获进程、文件、网络及加密通信明文等原始事件，写入 Redis 作为中间缓冲，解耦采集与分析的性能压力，同时支持上层多个检测模块的并发读取。

中层单智能体检测模块（Intra-Agent Monitor）：针对无交互或弱交互的单 Agent 场景（如 Claude Code、OpenClaw 等独立运行的端侧智能体），基于单一 Agent 的系统调用行为特征进行异常判定，重点关注资源滥用模式（高频 API 调用、重复 Prompt 请求伴随无意义文件操作）和安全违规模式（提示词注入、非预期 Shell 启动、敏感文件越权访问）。

中/高层多智能体检测模块（Inter-Agent Monitor）：针对交互式多 Agent 协作场景，基于 AutoGen 框架模拟两种典型协作关系进行异常监测：父子层级关系（Parent-Child）中父 Agent 拆解任务、子 Agent 执行指令，系统追踪指令传播链并识别子 Agent 偏离指令的异常行为；平级对等关系（Peer-to-Peer）中多个 Agent 平等协作，系统在进程交织环境中区分正常协作与异常竞争。

两个检测模块共享底层 eBPF 采集数据，通过 Redis 进行实时消费，各自独立运行异常检测逻辑，最终通过统一的告警输出模块生成含时间戳 PID/TID、操作对象、异常类型、时间序列、语义关联、根因定位、多 agent 交互视图等上下文的归因日志。

1.3 背景调研

1.3.1 现有方案概览

当前面向 AI Agent 的监测工具可大致分为两类：**应用层可观测平台**和**内核级运行时安全工具**，两者之间存在显著的观测盲区。

应用层工具以 LangSmith、Langfuse、Braintrust 为代表，通过 SDK 集成或 API 代理拦截 LLM 调用链，追踪 Prompt/Response、Token 用量和工具调用。这类工具在 Agent 决策链的可视化和质量评估方面较为成熟，但观测范围完全局限于应用语义层，无法感知 Agent 在操作系统层面的实际行为。此外，它们通常与特定框架（如 LangChain）强耦合，且需要设置环境变量或引入 SDK，存在一定侵入性。

内核级工具以 Falco（CNCF 毕业项目）和 Tetragon（Cilium 生态）为代表，通过 eBPF 捕获系统调用、文件操作和网络连接等底层事件。Falco 将所有事件传输至用户空间规则引擎匹配，开销较大；Tetragon 在内核内完成过滤，仅发射匹配策略的事件，开销显著更低。但两者均只关注安全事件本身，无法

关联到 Agent 的高层语义，它们能检测到“某个进程访问了敏感文件”，但不知道“这是哪个 Agent 在什么指令下执行的”。

学术界方面，ACM 2025 发表的 AgentSight 是目前最相关的工作，其核心创新在于边界追踪（Boundary Tracing）：在 Agent 与外界交互的稳定接口（内核 syscall 和网络通信）处进行监控，通过因果关联引擎将通信语义与内核事件关联。AgentSight 的思路与本方案最为接近，但它侧重于从加密流量的元数据中推定 Agent 意图，对多 Agent 协作拓扑（父子/平级）的区分和针对性异常检测未做深入探讨。

1.3.2 本方案的独特优势

与上述方案相比，本项目在以下维度具备差异化优势：

跨层关联而非单层观测：LangSmith/Langfuse 只看应用层，Falco/Tetragon 只看内核层。本项目通过 eBPF 采集内核事件并利用 uprobes 截获加密通信明文，贯通应用语义与底层行为的因果链路，弥补了两类工具之间的观测盲区。

结构化区分协作拓扑：现有的多 Agent 可观测工作将所有 Agent 协作视为统一场景进行追踪。本项目显式区分父子层级（Parent-Child）和平级对等（Peer-to-Peer）两种协作模式，针对各自的逻辑特征设计差异化的异常检测策略：父子模式下追踪指令传播链和子 Agent 偏离行为，平级模式下检测异常竞争和协作断裂。

零侵入且框架无关：应用层工具需要 SDK 集成或环境变量配置，且往往与特定框架耦合。本项目基于 eBPF 在内核态完成全部数据采集，无需修改 Agent 程序代码或重编译，对 Claude Code、OpenClaw、AutoGen 等不同框架均适用。

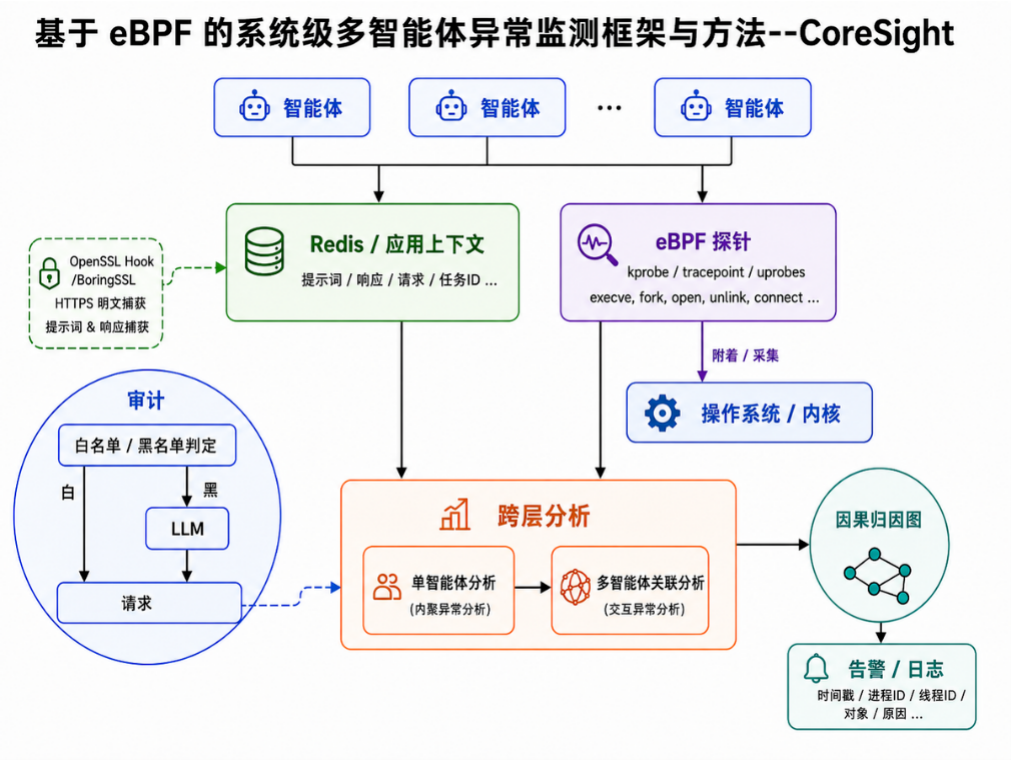
模块化的检测架构：本项目将异常检测拆分为 Intra-Agent（单智能体独立异常）和 Inter-Agent（多智能体协作异常）两个独立模块，通过共享的 Redis 数据层解耦，各自可独立迭代检测规则，相比 Falco/Tetragon 的单体规则引擎具备更好的扩展性。

本项目与其他项目对比见下表：

维度	应用层工具	内核安全工具	AgentSight	本项目
侵入性	需引入 SDK	零侵入	零侵入	**零侵入**
应用层语义观测	支持	不支持	部分支持	**支持**
内核态行为观测	不支持	支持	支持	**支持**
跨层因果关联	不支持	不支持	不支持	**支持**
多 Agent 协作异常检测	有限	间接	部分支持	**支持**
协作拓扑区分	不区分	不区分	不区分	**区分**
框架无关性	与特定框架耦合	无关	无关	**无关**

2 架构

2.1 架构图



2.2 整体架构说明

本项目整体采用“数据采集层-上下文关联层-跨层分析层-审计与输出层”的分层架构设计，核心目标是在不侵入智能体本身实现的前提下，同时贯通操作系统底层行为、应用层交互语义以及最终的异常审计结果。底层由 eBPF 探针负责对进程执行、文件访问、网络连接等系统调用进行实时采集，并通过 kprobe、tracepoint、uprobes 等机制获取运行时事件；同时结合 OpenSSL/BoringSSL Hook 捕获 HTTPS 明文中的提示词、响应内容、请求标识和任务标识等应用层上下文信息。两类数据分别进入 Redis 与应用上下文缓存层，在这一层完成异构数据的暂存、解耦和语义关联，为后续跨层分析提供统一输入。

在分析层，系统按照“单智能体异常”和“多智能体关联异常”两个方向进行模块划分。单智能体分析模块主要面向单个 Agent 的本地异常行为，重点识别异常命令执行、敏感文件访问、工作区越界、逻辑循环以及语义与控制异常；多智能体关联分析模块则面向多个 Agent 之间的协作关系，重点分析交互链

路、消息传播、异常传递和因果关联。这样的划分原则一方面把单体行为异常与协作异常分开处理，降低模块耦合度；另一方面保留跨层统一输入，使两个分析方向能够共享底层采集结果和应用上下文信息。

在审计与输出层，系统对分析模块生成的请求进一步进行黑白名单判定与 LLM 复核，形成更稳定的异常结论；随后再结合因果归因图对异常来源、传播路径和上下文证据进行组织，最终输出结构化告警与用户友好日志。整体数据流向可以概括为：智能体运行产生行为事件与交互上下文，事件分别进入 eBPF 采集链路和上下文捕获链路，经 Redis 与应用上下文层汇聚后送入跨层分析模块，再由审计模块完成确认，最后生成告警、日志和归因结果。这样既保证了底层监测的实时性，也增强了上层异常解释与结果呈现的完整性。

2.3 模块职责一览

模块名称	职责描述	对应源码目录/文件
多层级数据捕获	利用 eBPF 监控多智能体并发运行时的底层系统调用行为，包括资源使用情况、进程管理、文件系统操作以及网络通信；获取智能应用层交互信息，非侵入式地截获提示词 Prompt/回答 Response 明文文本。	ebpf-multi-agent-monitor/src/bpf/* ebpf-multi-agent-monitor/src/user/*
intra-agent	对单个智能体的系统调用与 I/O 行为做滑动窗口统计和规则检测，识别敏感文件访问、工作区越界删除、异常 shell 启动、逻辑死循环等单体异常。	ebpf-multi-agent-monitor/src/analysis/intra_agent/main.py； ebpf-multi-agent-monitor/src/analysis/main_analyzer.py
inter-agent	实现对多智能体系统的实时异常检测与响应，实时检测分布式智能体在两种协	ebpf-multi-agent-monitor/src/analysis/inter_agent/*

	作过程中的行为异常与通信异常。	
归因定位	通过时间窗口和智能体 id 对应用层交互信息与内核层系统行为建立跨层的数据关联，基于进程关系及附带交互信息对识别到的异常行为执行链路进行根因定位。	ebpf-multi-agent-monitor/src/analysis/main_analyzer.py ebpf-multi-agent-monitor/src/analysis/inter_agent/main.py ebpf-multi-agent-monitor/src/analysis/intra_agent/main.py ebpf-multi-agent-monitor/src/analysis/audit_engine.py ebpf-multi-agent-monitor/src/config/*
CoreSight showcase	对主监控项目落库结果进行只读聚合与可视化展示，提供总览指标、异常审计结果查看及对话式追溯能力。	ebpf-monitor-showcase/app/*； ebpf-monitor-showcase/components/*； ebpf-monitor-showcase/lib/server/*

3 设计思路

3.1 核心设计决策

Agent 通信的流量捕获：系统能够在单 Agent 场景和多 Agent 交互场景进行 Prompt/Response 数据捕获。经过前期调研，对于智能体与大模型通信，目前存在的通信方式包括四种：标准 OpenSSL 路径（Python Agent）、BoringSSL 路径（Node.js Agent）、OpenSSL_ex 路径（现代大模型 SDK）、Go TLS 路径（Go Agent），因此我们使用 uprobe 分别挂载在对应的位置并采用对应的挂载方式进行明文捕获。对于 Agent 之间通信，现有的通信方式包括三种：MCP（stdio Agent）、A2A/ACP/ANP（localhost HTTP agent）、gRPC (HTTP/2 protobuf Agent)，我们使用 kprobe 和 tracepoint 对系统调用进行监控，获取智能体间通信数据。

采用 Redis 作为数据采集层与异常分析层的中间层：Redis 支持对同一个逻辑实体在一组事件循环中叠加多种数据结构指令，对于在复杂的拓扑场景下的保存 Agent 及其通信链路特征 Redis 能够有效免除传统数据库的多表关联负担与 Schema 定义开销。Redis 中的 SADD 命令具备幂等性，对于在有交互的多 Agent 场景下的构建 Agent 之间交互关系能够有效防止同一条边被重复构建。

单智能体检测与审计分层机制：intra-agent 并不直接把规则命中等同于最终告警，而是采用“规则快速筛查 + 黑白灰名单分流 + LLM 语义复核”的分层设计。前端规则层围绕进程执行、文件访问、目录删除、网络连接和标准输入输出等行为维护滑动窗口统计，先快速识别异常命令执行、敏感文件访问、工作区边界越界、逻辑循环以及语义控制异常等高风险信号。随后审计层根据 rules.yaml 中的黑名单、白名单和灰名单配置进行分流：白名单事件直接放行，灰名单事件保留上下文进入关联分析，黑名单事件进入 LLM 二次复核。这样设计的原因在于，规则检测适合做高性能前置筛查，但复杂 Agent 行为往往需要结合任务语义、触发对象和上下文证据进一步判断；LLM 复核则用于输出结构化的 decision、risk_level、reasoning 和 evidence，使最终告警不仅能说明“命中了什么规则”，也能解释“为什么这次行为确实有风险”。

多检测器异构融合架构：系统通过 6 个异构检测器流水线编排，而非单一模型方案，核心在于覆盖不同粒度的异常模式。SemanticRuleChecker 覆盖 15 种结构化语义规则，MessageAnomalyDetector 通过基线+阈值检测消息流异常，AnomalyDetector (DOMINANT) 以 GNN 重构误差发现无预设类型的结构性异常，TemporalAnomalyDetector 通过滑动窗口 z-score 感知时间维度的渐进偏移，PropagationPathAnalyzer 以 BFS 追踪异常传播路径。各检测器输出经加权融合并保底提升，既保留强信号的立即响应，又通过组合避免单检测器盲区。创新点在于将规则系统、图神经网络与时序统计在统一框架内解耦编排，无需 LLM 即可覆盖 19 种预定义类型，且大幅提升检测效率。

3.2 技术选型

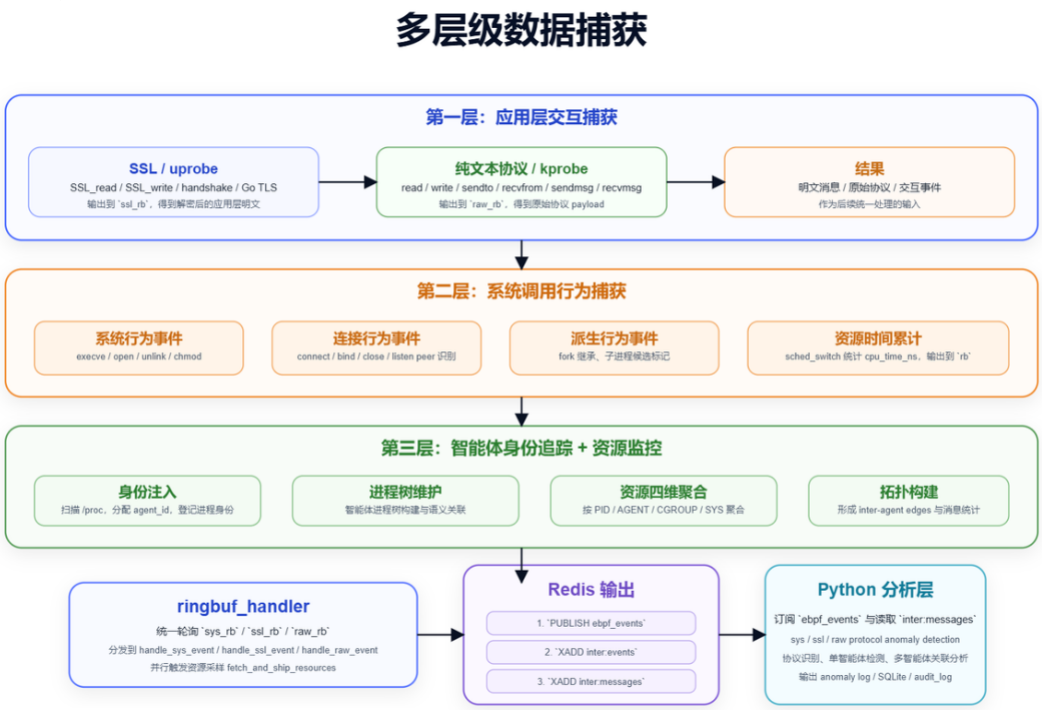
技术/工具	选型理由
libbpf	支持 CO-RE，具有良好兼容性和可移植性；运行时轻量化，对性能损耗小；提供了对 Ring Buffer 高效数据传输机制的支持，适用于当前内核向用户空间传递海量事件数据的需求
redis	redis 具有优秀的读写性能，适合在抓取数据与分析数据间作为缓存层，保障系统快速响应智能体异常；redis 支持丰富的数据类型，能够兼容抓取到的多种类型数据；redis 可实时推送事件，能够及时识别异常；redis 可持久化，支持回查，能够进行归因定位和构建多 agent 场景的拓扑图
黑白名单+LLM 机制	黑白名单机制能够对高风险和低风险事件进行快速分流，适合处理敏感文件访问、异常 Shell 启动、工作区越界等规则边界清晰的行为，具有响应快、开销低、可解释性强的优势；在此基础上引入 LLM，对逻辑循环、语义与控制异常等难以仅靠固定规则准确判断的场景进行二次复核，能够结合上下文理解异常行为的真实含义，降低误报率并提升结论质量。两者结合后，既保留了规则方法在实时性和稳定性上的优势，又补充了大模型在复杂语义判断上的能力。
异构多检测器流水线	GNN 适合对多智能体交互关系、传播路径和上下文依赖进行统一建模，能够保留节点与边的拓扑结构信息；通过带标签样本学习正常模式与异常模式的差异，可识别规则方法难以覆盖的复杂协作异常和隐含关联特征，适用于本项目中 inter-agent 场景下的异常检测任务。利用组合模型（语义规则检查器+消息异常

	检测器+传播路径分析器+时间异常检测器+GNN 检测器+协调器）对多智能体间所有交互行为记录风险分数，对超过安全阈值的行为进行告警。

4 实现描述

4.1 多层级数据捕获

4.1.1 功能描述



在零侵入的前提下，对宿主机上运行的多 AI Agent 系统进行数据采集，将应用层语义的 Prompt/Response 与内核层系统调用层行为关联到统一的智能体进程树上。RingBuf 通过/sys/fs/bpf/钉固，ringbuf_handler 程序通过 bpf_obj_get 跨进程消费，事件序列化为 JSON 后推入 Redis。

4.1.2 实现细节

1. 子智能体识别

在多智能体协作场景下，经典架构可分为父子层级关系（Parent-Child / Hierarchical）与平级对等关系（Peer-to-Peer / Joint）两种典型多智能体协作拓扑，在父子层级关系中，子智能体由父智能体 fork 产生。为区分子智能体与常规子进程便于后续分析处理，我们需要构建子智能体识别部分。

子智能体的判别分为两步：判定是否是由父智能体 fork 产生的进程；判断进程是否为智能体。对此，我们为进程设计了三个状态：通过父智能体 fork 检查，待确认为智能体进程 CANDIDATE_SUB_AGENT，已被识别为智能体的进程 CONFIRMED_SUB_AGENT，被判定为非智能体的进程 NORMAL_TOOL。

SEC("tracepoint/sched/sched_process_fork") 挂载在内核 sched_process_fork tracepoint 上。当任何进程调用 fork()/clone()时，内核在创建子进程后触发此 tracepoint，此时我们将该进程智能体标识设置为候选，完成第一步判定。fork 仅复制了父进程的地址空间，此时子进程的 comm 仍为父进程的名称，无法判断其真实身份。子进程执行 execve 加载新程序映像，内核将 comm 更新为真实程序名，同时 filename 参数提供完整路径。eBPF 在 ksyscall/execve 处捕获此事件，将 comm、filename 与已从 fork 钩子继承的智能体信息提交给用户态。

对于第二步判定，我们洞察到需要特殊分析的智能体进程与普通进程的关键区别在于智能体进程会调用 AI API 或产生智能体间通信协议流量。为此我们设计 SEC("ksyscall/connect") 挂载在内核__x64_sys_connect 入口，覆盖现有大多数 AI API 调用默认端口，保证在 API 调用行为被抓取；对于智能体间通信协议流量，eBPF 通过 kprobe 挂载 read/write 系统调用捕获 MCP stdio 流量，并通过挂载 sendto/recvfrom/sendmsg/recvmsg 捕获本地 TCP 智能体间通信流量，对捕获数据进行固定字节序列匹配，判定是否可能为智能体间交互信息。为区分普通网络连接与 API 调用和智能体间通信，我们对 comm 进行过滤，保证 curl、wget 等不会被误判。

2. Agent 与大模型通信明文捕获

经过前期调研，我们针对不同类型的智能体进行不同处理方式，对 uprobe 探针实现四种手动挂载以适配不同类型 Agent。挂载方式说明如下表：

挂载方式	适配智能体	挂载逻辑
OpenSSL 动态库	Python、C/C++、Ruby 等动态链接语言编写的 Agent 依赖操作系统底层 libssl.so 动态链接库进行 HTTPS 通信。 (测试对象 AutoGen)	扫描/proc/<pid>/maps 中 libssl.so 路径，按符号名 SSL_write/SSL_read/SSL_write_ex/SSL_read_ex 挂载 uprobe/uretprobe 到共享库
BoringSSL 字节码偏移	商业化独立客户端或重度定	用 mmap 载入 ELF 二进

	制底层的 Agent，这类应用出于跨平台、体积优化或防逆向的考量，将 BoringSSL 静态内置编译到程序中，并且彻底抹除了函数符号表。 (测试对象 Claude)	制，在.text 段扫描函数前导字节码定位 SSL_read/SSL_write 入口，再计算相对偏移，按物理偏移量挂载
Go TLS	Golang 编写的云原生 Agent Go 语言在编译时是完全静态链接的，且使用的是其标准库中自研的 TLS 实现。 (测试对象 LangChainGo)	按 Go ABI 符号 crypto/tls.(*Conn).Write/Read 挂载；Write 为同步调用，enter 从 BX/CX 寄存器直接读，无需 uretprobe
Node 默认符号	基于 Node.js / JavaScript / TypeScript 编写的 Agent Node.js 默认将 OpenSSL 库静态编译到了 node 可执行文件中，挂载系统 libssl.so 无效。 (测试对象 LangChain.js)	在目标二进制自身按符号名 SSL_write/SSL_read 尝试挂载；适用于 OpenSSL 静态链接进二进制、但仍保留 .symtab 符号表

3. 数据封装

采集到的内核事件与明文载荷在用户态被统一封装为六类结构化数据，通过 Redis 多种数据类型进行推送，兼顾实时性、持久性、可查询性与跨语言消费能力。事件源通过 3 个 pinned ringbuf 即 sys_fd、ssl_fd、raw_fd 汇聚到 ringbuf_handler，经类型判别后分别调用 ship_sys_event、ship_ssl_event、ship_raw_event 等函数，最终以 redisCommand 写入 Redis，用以上层分析层进行异常检测和根因定位。

数据类型	数据内容	设计思路	抓取方式

实时事件流	JSON 字符串：type, pid, ppid, uid, timestamp, comm, filename, dip, dport, agent_id, parent_agent_id, peer_agent_id, main_pid, depth, is_sub_agent, data(base64)	低延迟 pub/sub 通道，订阅方实时消费无需轮询；JSON 自描述兼容任意语言消费者；base64 编码绕过 Redis 二进制安全限制	ringbuf_handler 从 sys_fd/ssl_fd/raw_fd 轮询
系统行为事件流	field-value 对：event_type, subtype, timestamp_ns, pid, tid, comm, filename, daddr, dport, agent_id, parent_agent_id, parent_agent_name, peer_agent_id, peer_pid, peer_agent	Stream 支持持久化和消费者组	eBPF kprobe、tracepoint 捕获 execve/fork/connect/open/unlink/chmod
消息载荷流	field-value 对：msg_id, src, dst, type, protocol, timestamp_ns, pid, tid, main_pid, comm, agent_id, bytes, data	msg_id 单调递增确保有序性；src/dst 支持拓扑重建；data 保存 JSON 转义后的明文载荷，可直接查看	eBPF uprobe SSL_read/SSL_write 拦截明文 buf eBPF kprobe read/write/sendto/recvfrom 捕获 buf
智能体元数据	Set：agent_id 集合	Set 提供 O(1) 成员	agent 识别过程中维

	Hash：agent_id, parent_agent_id, main_pid, comm, workspace, depth, is_sub_agent, status, group	判定和全量遍历；parent_agent_id 和 depth 支持智能体进程树信息查询	护元数据信息
通信拓扑边	Set：src:dst 边集合 Hash：src, dst, type, weight, protocol, daddr, dport, last_seen_ns, total_messages, total_bytes	Set 去重，遍历所有边；src:dst 支持有向图；HINCRBY total_messages/total_bytes 无锁累加通信量，避免重复写入	connect 事件触发边创建，后续消息累加计数
四级资源指标	Hash：agent_id, level, timestamp_ns, cpu_time_ns, mem_rss_bytes, pid_count, open_fds	支持 pid、agent、cgroup、sys 级别资源占用数据获取；cpu_time_ns 和 mem_rss_bytes 反映智能体资源消耗	eBPF 侧通过 kprobe finish_task_switch 跟踪 CPU 时间，定时更新 map

4.1.3 核心代码片段

代码片段 1：Go 编写的 Agent 明文捕获。代码片段通过宏定义屏蔽了底层 CPU 架构的差异，定位 Go 的 TLS 读写函数参数在寄存器中的位置，捕获收发的明文数据。

```
#if defined(__TARGET_ARCH_x86)
        #define GO_BUF_REG      regs->bx
        #define GO_LEN_REG      regs->cx
        #define GO_RET_REG      regs->ax
#elif defined(__TARGET_ARCH_arm64)
        #define GO_BUF_REG      regs->x0
        #define GO_LEN_REG      regs->x1
        #define GO_RET_REG      regs->x0
#else
        #define GO_BUF_REG      regs->bx
        #define GO_LEN_REG      regs->cx
        #define GO_RET_REG      regs->ax
#endif

SEC("uprobe/go_tls_write")
int BPF_UPROBE(go_tls_write_enter) {
        u64 pid_tgid = bpf_get_current_pid_tgid();
        if (!trace_allowed(bpf_get_current_uid_gid(), pid_tgid >> 32)) return
0;

        struct pt_regs *regs = (struct pt_regs *)ctx;
        void *buf = (void *)GO_BUF_REG;
        int len = (int)GO_LEN_REG;

        u64 ts = bpf_ktime_get_ns();
        u32 tid = (u32)pid_tgid;
        bpf_map_update_elem(&start_ns, &tid, &ts, BPF_ANY);
        process_data_exit(buf, len, 1, false, ENGINE_GO_TLS);
        return 0;
```

代码片段 2：子智能体识别。代码片段通过识别通信行为与过滤普通通信进程，判定已经被识别为由父级智能体 fork 的子进程是否为子智能体。

```
static __always_inline bool promote_to_sub_agent(__u32 pid) {
    struct process_context *child_ctx = bpf_map_lookup_elem(&monitored_pids, &pid);
    if (!child_ctx) return false;
    if (child_ctx->is_sub_agent != CANDIDATE_SUB_AGENT) return false;
    if (is_known_tool_comm()) return false;

    child_ctx->is_sub_agent = CONFIRMED_SUB_AGENT;
    child_ctx->agent_id = AGENT_ID_AUTO_DETECT(pid);
    child_ctx->depth = 1;
    return true;
}
```

4.2 intra-agent

4.2.1 功能描述



intra-agent 模块负责对单个智能体在运行过程中的异常行为进行实时检测，是系统跨层分析链路中的第一道细粒度筛查环节。该模块接收主分析器转发的

标准化事件数据 event_data，围绕同一智能体对应的进程执行、文件访问、目录删除、网络连接、加密通信明文收发以及标准输入输出行为，构建单体行为视图，并输出待审计请求 req。

与 inter-agent 关注多智能体之间的协作拓扑和传播链路不同，intra-agent 更强调“单个 Agent 自身是否出现异常失控”。其检测目标主要包括五类：

1. **异常命令执行：**例如智能体非预期地启动 bash、sh 等 shell 解释器。
2. **敏感文件访问：**例如读取或删除/etc/passwd、/.ssh/、.env 等高风险对象。
3. **工作区边界越界：**例如对工作目录之外的系统路径执行删除或改写。
4. **逻辑失控或资源空转：**例如在短时间内反复进行高频文件或网络操作但对象高度集中，表现出明显的循环调用特征。
5. **语义与控制异常：**例如违反任务规范、违反角色规范、步骤重复、对话历史丢失以及未识别终止条件等由语义偏移或控制失效引发的异常行为。

该模块的输出不是最终告警，而是待审计请求。虽然单靠规则可以先筛选出明显异常行为，但遇到复杂场景时，还需后续审计模块继续判断，才能降低误报率。

4.2.2 实现细节

intra-agent 模块的核心实现位于 IntraAgentAnalyzer 类中。模块以 PID 为索引维护 agent_stats 状态表，为每个正在观测的智能体进程记录窗口起始时间、最近告警时间、行为计数器以及文件访问样本集合等上下文信息。当前实现采用 30 秒滑动窗口来统计单体行为，并针对不同异常规则配置独立冷却时间，从而避免短时间内重复触发同类告警造成日志刷屏。

在状态管理层，模块对 proc_exec_cnt、fs_open_cnt、fs_unlink_cnt、net_connect_cnt、ssl_read_cnt、ssl_write_cnt、stdio_read_cnt、stdio_write_cnt、net_tcp_send_cnt、net_tcp_recv_cnt 等计数器进行统一维护，并使用 deque 保存最近访问过的文件样本。每当新事件到来时，模块首先根据时间戳判断是否进入新的统计窗口；如果超过窗口时长，则清空对应窗口内的计数器和样本，重新开始统计。

在规则检测层，模块按事件类型分支处理：对于 EVENT_EXECVE，如果执行目标属于 bash、sh、zsh、dash 等 shell 解释器集合，则触发 UNEXPECTED_SHELL_EXEC 规则；对于 EVENT_OPEN 和 EVENT_UNLINK，如果目标路径命中/etc/passwd、/etc/shadow、/.ssh/、/.aws/credentials、.env 等敏感标记，则触发 SENSITIVE_FILE_ACCESS 规则；对于 EVENT_UNLINK，如果删除路径既不是工作区路径，也不在/tmp 或/var/tmp 等临时目录中，则触发 OUT_OF_WORKSPACE_OPERATION 规则；对于 EVENT_CONNECT，如果外连端口不是常见的 80 或 443，则生成 UNUSUAL_NETWORK_CONNECTION 信号，供后续审计模块结合上下文进一步判断。

此外，模块还实现了一个基于统计特征的逻辑循环检测机制。系统将文件类操作计数与网络类操作计数分别聚合，当二者在同一时间窗口内都达到较高阈值、且涉及的文件对象数量很少时，说明智能体可能陷入“对少量对象重复读写并频繁发起请求”的循环模式。此时模块会触发 POSSIBLE_LOGIC_LOOP 规则，用于发现提示注入、错误规划或终止条件识别失败导致的资源浪费行为。

整体上，intra-agent 模块只负责单体视角下的初步筛查，先用规则和统计特征把可疑行为挑出来，再交给后续模块继续处理。这样做主要是为了把前段检测保持得更直接一些，不把太多复杂判断都压在这一层。

4.2.3 核心代码片段

代码片段 1：滑动窗口状态维护。该片段展示模块如何按 PID 管理状态，并在窗口超时后清空统计计数器，为后续的循环检测与高频行为判定提供基础。

```
    def _get_stats(self, pid, ts_ns, comm):
        s = self.agent_stats.get(pid)
        if s is None:
            s = self._new_stats(pid, ts_ns, comm)
            self.agent_stats[pid] = s
        if ts_ns - s["window_start_ns"] >= self.loop_window_ns:
            s["window_start_ns"] = ts_ns
            s["proc_exec_cnt"] = 0
            s["fs_open_cnt"] = 0
            s["fs_unlink_cnt"] = 0
            s["net_connect_cnt"] = 0
            s["net_tcp_send_cnt"] = 0
            s["net_tcp_recv_cnt"] = 0
            s["ssl_read_cnt"] = 0
            s["ssl_write_cnt"] = 0
            s["stdio_read_cnt"] = 0
            s["stdio_write_cnt"] = 0
            s["unique_files"].clear()
        return s
```

代码片段 2：关键规则匹配。该片段展示了对进程执行、敏感文件访问和工作区外删除三类典型异常的检测方式。

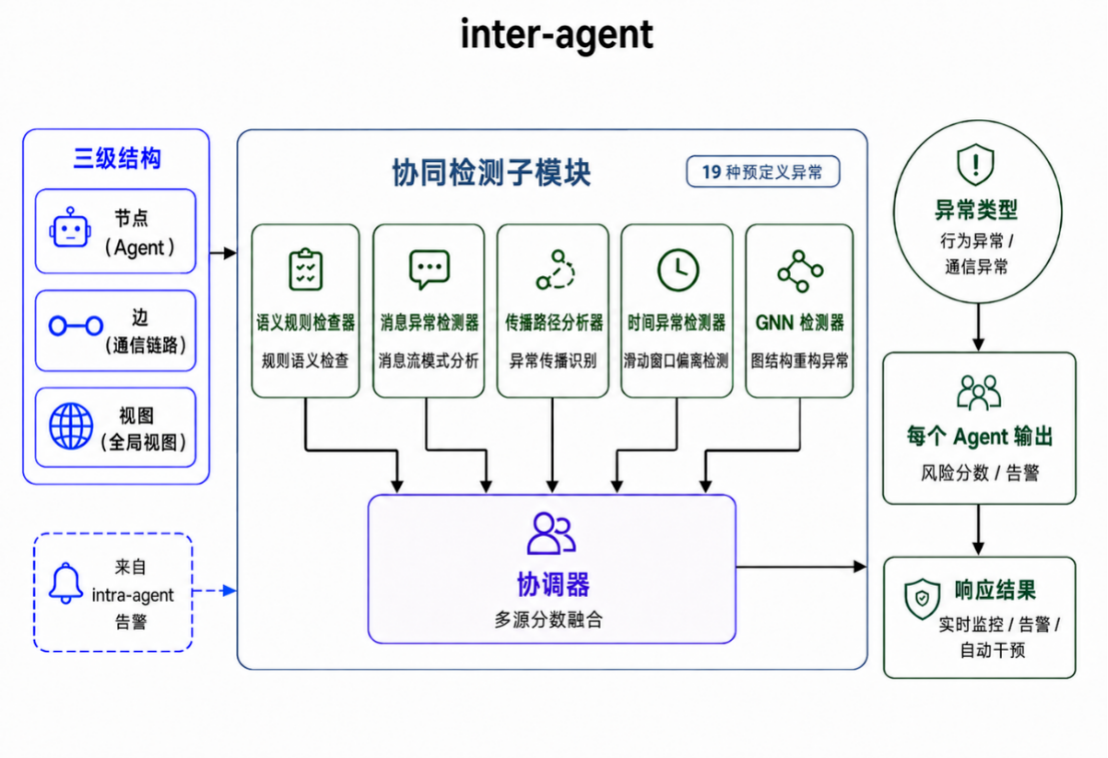
```
if event_type == EVENT_EXECVE:
        s["proc_exec_cnt"] += 1
         if self._is_shell_exec(path) and ts_ns - s["last_shell_alert_ns"] >= self.event_alert_cooldown_ns:
            s["last_shell_alert_ns"] = ts_ns
            reqs.append(
                self._build_req(
                    event_data,
                    "UNEXPECTED_SHELL_EXEC",
                    path,
                    "检测到进程执行 shell 解释器",
                    "HIGH",
                )
            )
elif event_type == EVENT_OPEN:
        s["fs_open_cnt"] += 1
        self._remember_file(s, path)
        if self._path_is_sensitive(path) and ts_ns - s["last_sensitive_alert_ns"] >= self.event_alert_cooldown_ns:
            s["last_sensitive_alert_ns"] = ts_ns
            reqs.append(
                self._build_req(
                    event_data,
                    "SENSITIVE_FILE_ACCESS",
                    path,
                    "检测到敏感路径访问",
                    "HIGH",
                )
            )
```

代码片段 3：逻辑循环检测。该片段体现了系统如何综合文件类操作、网络类操作以及对象集中度，识别可能的逻辑失控行为。

```
def _maybe_loop_req(self, event_data, s):
    io_cnt = s["fs_open_cnt"] + s["fs_unlink_cnt"] + s["stdio_read_cnt"] + s["stdio_write_cnt"]
    net_cnt = s["net_connect_cnt"] + s["ssl_read_cnt"] + s["ssl_write_cnt"] + s["net_tcp_send_cnt"] + s["net_tcp_recv_cnt"]
    ts_ns = int(event_data.get("timestamp", 0))
    if io_cnt < 120 or net_cnt < 120:
        return None
    if len(s["unique_files"]) > 3:
        return None
    if ts_ns - s["last_loop_alert_ns"] < self.loop_alert_cooldown_ns:
        return None
    s["last_loop_alert_ns"] = ts_ns
    return self._build_req(
        event_data,
        "POSSIBLE_LOGIC_LOOP",
        "-",
        "文件与网络操作高频且对象集中，疑似逻辑循环",
        "MEDIUM",
    )
```

4.3 inter-agent

4.3.1 功能描述



inter-agent 模块是多智能体系统的实时异常检测与响应框架，专注于检测分布式智能体在协作过程中的行为异常与通信异常。

本模块以**“节点（Agent）、边（通信链路）、视图（全局限）”**三级结构组织检测逻辑，通过 6 个协同工作的检测子模块（语义规则检查器、消息异常检测器、传播路径分析器、时间异常检测器、GNN 检测器、协调器）对 19 种预定义异常类型进行综合判别。各子模块分别从语义一致性、消息通信模式、传播路径、时序行为、图结构重构等多个维度独立计算风险分数，最终由协调器进行加权融合，为每个 Agent 输出组合风险分数和告警，实现对分布式智能体系统的实时监控、异常告警与自动干预。

各子模块职责如下：

1. **语义规则检查器：**基于预定义规则的语义异常检查模块。维护一个包含 15 条规则的注册表（11 种语义规则+4 种拓扑规则），对每条观测逐规则检查，计算每个 agent 在每种异常类型下的风险分数，并汇总输出。
2. **消息异常检测器：**基于消息流触发器的异常检测模块。统计 agent 间消息频率、级联程度、数据量，结合来自 intra-agent 的告警，产生 4 种内部异常模式并对风险分数做贡献。

3. **传播路径分析器：**基于 BFS 的跨 agent 传播路径识别模块。以被标记为异常的节点为起点，沿交互图进行广度优先搜索，识别异常在 agent 间的传播路径和受影响范围。
4. **时间异常检测器：**基于滑动窗口的时间异常检测模块。不产生新的异常类型，仅作为分数提升器。提取 4 维特征构建时间序列，使用 z-score 检测各 agent 的显著偏离。
5. **GNN 检测器：**包含两个子检测器，RuleBasedDetector 基于阈值产生原始风险分数；AnomalyDetector（DOMINANT）基于图神经网络重构节点属性和图结构，计算异常分数，是无类型异常分数器，可以发现结构性异常。
6. **协调器：**系统总协调模块。依次调用 6 个检测器，将各模块结果按加权公式融合，结合消息异常分数提升，为每个 agent 输出组合风险分数及决策响应级别。

4.3.2 实现细节

系统共包括 6 个检测模块，共同对 19 种已命名异常类型起检测作用，整体以“节点、边、视图”的三级结构组织。

1. **SemanticRuleChecker（语义规则检查器）：**基于规则的语义异常检查模块，规则注册表中有 15 种异常如下：

异常类型	级别	检测逻辑
stale_state（过时状态）	node	fs_unlink_count>=2+消息包含陈旧关键字
state_conflict（状态冲突）	edge	两个或更多 agent 的 fs_write_count>20（正常最大值=20）
partial_visibility（局部可见性）	node	fs_open_count<=2 且 unique_files_count<=3

message_disorder（消息乱序）	edge	回复的时间戳在请求的时间戳之前（因果顺序错误）
timeout_retry（超时重试）	node	api_call_count>30 或 fs_write_count>25+消息中的重试关键字
ignore_input（忽略输入）	edge	收到的消息数>=已发送消息数+1（收到但未回复）
reasoning_action_mismatch（推理与行动不匹配）	node	shell_exec_count>0 或 sensitive_file_count>0（正常值始终为 0）
wrong_assumption（错误假设）	node	net_connect_count==0 且 net_send_count==0（正常值始终非零）
task_derailment（任务偏离）	node	proc_exec_count >= 4 且 api_call_count >= 20
concealment（隐瞒行为）	node	cpu_usage > 0.85 且 memory_usage > 0.85
premature_convergence（过早收敛）	node/view	api_call_count <= 3 且 fs_read_count <= 5（过早达成一致）
supervisor_bypass（越级通信）	edge	任意交互 type=="bypass"且 src 和 dst 均在 agent 集合中
group_split（组分裂）	edge/view	agent 跨组交互数>= min_cross，且平均权重<0.3

flood_from_single（单源洪泛）	edge/view	单个 src 频繁对不同 dst 发送交互或消息
message_storm（消息风暴）	edge/view	msg_received>=30 且 incoming>=5

2. **MessageAnomalyDetector（消息异常检测器）：**基于触发器的消息异常检查模块，对风险分数起作用，从而触发如 resource、security、message_storm 等类型。4 种内部异常模式如下：

异常类型	级别	检测逻辑
frequency_surge（频率激增）	edge	src→dst 消息计数值超过均值+2.5 倍标准差
cascade_storm（级联风暴）	edge	dst 在窗口内从许多源收到消息（>=cascade_threshold，默认 5）
data_size_anomaly（数据量异常）	edge	消息负载大小超过阈值
intra_agent_alert（内部 Agent 告警）	bridge	来自 intra-agent 检测器的 POSSIBLE_LOGIC_LOOP/ UNEXPECTED_SHELL_EXEC 规则

3. **PropagationPathAnalyzer（传播路径分析器）：**基于 BFS 的跨 agent 交互图的传播路径识别模块，主要对所有 agent 间路径进行检测、识别、标记，精准检测到异常节点以及异常 src→dst 对，便于追根溯源。

4. **TemporalAnomalyDetector（时间异常检测器）：**基于时间窗口的时间异常检查模块，不产生新类型的分数提升器，使用 4 维特征向量（cpu、memory、api_call_count、总消息数）构造滑动窗口时间历史，使用 z-score 检测与近期均值的显著偏差，为每个 agent 产生 temporal_scores（风险 0.0-1.0），当其分数>risk_threshold 时标记为 anomaly_agents。

5. **GNN 检测器：**包含 2 个检测器（RuleBasedDetector 和 AnomalyDetector /DOMINANT）。RuleBasedDetector 是基于阈值的分数计算器，在 19 种类型系统之外产生原始风险分数，进行等效类型映射，不产生新类型；AnomalyDetector 是基于 GNN 的图级异常检测器，使用 DOMINANT（深度图异常检测模型），配备 GCN 或 GAT 编码器，重构节点属性和图结构，每个节点根据 0.5×(feature_weight×feat_err+struct_weight×struct_err[+ edge_weight×edge_err])+0.5×cls_probs 计算异常分数，分数>阈值即标记为异常，DOMINANT 是一个无类型异常分数器，**可以发现任何结构性异常，无需预定义类型**。

6. **MonitorAgent（协调器）：**对所有模块进行加权融合，对每个 agnet 得出组合系统的风险分数。组合系统当前可检测的预定义异常类型如下：

级别	异常类型
node	resource（CPU/内存超过阈值） security（shell 执行/敏感文件访问） worker_isolated（worker 与 supervisor 隔离） reasoning_action_mismatch（声称安全但执行了 shell） wrong_assumption（假设有网络但无连接） task_derailment（高进程执行+API 调用表明偏离任务） concealment（高 CPU/内存但与良性消息内容矛盾） premature_convergence（在最小工作后过早达成一致） stale_state（高文件删除+陈旧缓存消息） partial_visibility（文件可见性低） timeout_retry（高 API/写入+消息中的重试

	关键字）
edge	state_conflict（多个 agent 并发写入同一资源） message_disorder（消息因果顺序违反） ignore_input（收到消息但未回复） group_split（agent 组在网络分区中分裂） supervisor_bypass（绕过 supervisor 直接访问 worker） message_storm（许多 src agent 向一个 dst agent 发送消息） flood_from_single（单个 agent 向所有其他 agent 洪泛） propagation（agent 间的级联异常传播）
view（与 node/edge 重叠）	propagation（edge/view） group_split（edge/view） message_storm（edge/view） flood_from_single（edge/view） premature_convergence（node/view）

4.3.3 核心代码片段

代码片段 1：SemanticRuleChecker 规则迭代与证据提取。该片段展示规则注册表中 15 条规则的统一调度方式，check()方法遍历 RULES 列表，逐条调用规则函数收集风险分数并汇总。

```
RULES: List[Tuple[str, Callable]] = [
    ("stale_state",        _rule_stale_state),
    ("state_conflict",    _rule_state_conflict),
    # ... 共 15 条（11 语义 + 4 拓扑）
    ("message_storm",     _rule_message_storm),
]
def check(self, obs: dict) -> dict:
    all_scores, types_found = {}, {}

    for type_name, rule_fn in self.RULES:
        per_agent = rule_fn(agents, messages, interactions, agent_ids)
        for aid, risk in per_agent.items():
            all_scores[aid] = max(all_scores.get(aid, 0.0), risk)
            types_found.setdefault(type_name, {})[aid] = risk
    return {
        "agents": all_scores,
        "types": types_found,
        "overall_risk": max(all_scores.values()) if all_scores else 0.0,
        "anomaly_types": list(types_found.keys())
    }
```

代码片段 2：MessageAnomalyDetector 四种异常检测。该片段展示基于消息流的触发式检测，分别对频率、级联程度、数据体量和来自 intra-agent 的内部告警做出风险评分。

```
def detect(self, obs: dict) -> dict:
    pair_count, cascade_count, pair_size = defaultdict(int), defaultdict(int), defaultdict(float)
    for m in obs.get("messages", []):
        src, dst = m["src"], m["dst"]
        pair_count[(src, dst)] += 1
        cascade_count[dst] += 1
        pair_size[(src, dst)] += float(m.get("data_size", 0))

    results, vals = [], list(pair_count.values()) or [0]
    mean, std = np.mean(vals), np.std(vals) + 1e-8

    # 频率突增检测
    for (s, d), c in pair_count.items():
        if c > mean + 2.5 * std:
            results.append({"type": "frequency_surge", "src": s, "dst": d,
                            "risk": min((c - mean - 2.5 * std) / (std * 3), 1.0)})
    # 级联风暴检测
    for d, c in cascade_count.items():
        if c >= 5:
            results.append({"type": "cascade_storm", "src": "*", "dst": d,
                            "risk": min(c / 10, 1.0)})
    # 数据量异常检测
    for (s, d), b in pair_size.items():
        if (mb := b / 1_048_576) > 100.0:
            results.append({"type": "data_size_anomaly", "src": s, "dst": d,
                            "risk": min(mb / 200, 1.0)})
    # Intra-agent 告警接入
    for a in obs.get("anomalies", []):
```

代码片段 3：PropagationPathAnalyzer BFS 传播路径。该片段展示以异常节点为起点、沿交互图广度优先搜索传播路径的核心算法。

```
def _bfs_propagate(self, source: str, anomalous_nodes: list) -> list:
    paths, visited, queue = [], {source}, deque([(source, [source], 0)])
    anom_set = set(anomalous_nodes) | {source}
    while queue:
        cur, path, depth = queue.popleft()
        if depth >= self._max_hops:
            continue
        for nbr in self._graph.successors(cur):
            if nbr in visited:
                continue
            visited.add(nbr)
            new_path = path + [nbr]
            paths.append({
                "path": new_path,
                "hop": depth + 1,
                "source": source,
                "target": nbr,
                "is_anomalous": nbr in anom_set
            })
            if nbr in anom_set and depth + 1 < self._max_hops:
                queue.append((nbr, new_path, depth + 1))
    return paths
```

代码片段 4：TemporalAnomalyDetector Z-Score 计算。该片段展示基于 4 维滑动窗口的 z-score 异常检测，计算当前特征向量与历史均值的统计偏离程度并归一化。

```
def detect(self) -> dict:
    scores = {}
    for aid, hist in self._history.items():
        arr = np.array(hist)
        if len(arr) < 3:
            scores[aid] = 0.0
            continue
        mean, std = np.mean(arr, axis=0), np.std(arr, axis=0) + 1e-8
        risk = float(np.mean(np.abs((arr[-1] - mean) / std)) / 5.0)
        scores[aid] = min(risk, 1.0)
    return {"temporal_scores": scores,
            "anomaly_agents": [a for a, s in scores.items() if s > self._risk_threshold]}
```

代码片段 5：AnomalyDetector (DOMINANT) 组合评分。该片段展示 DOMINANT 模型将结构重构误差与分类器概率加权融合得到最终异常分数的方法。

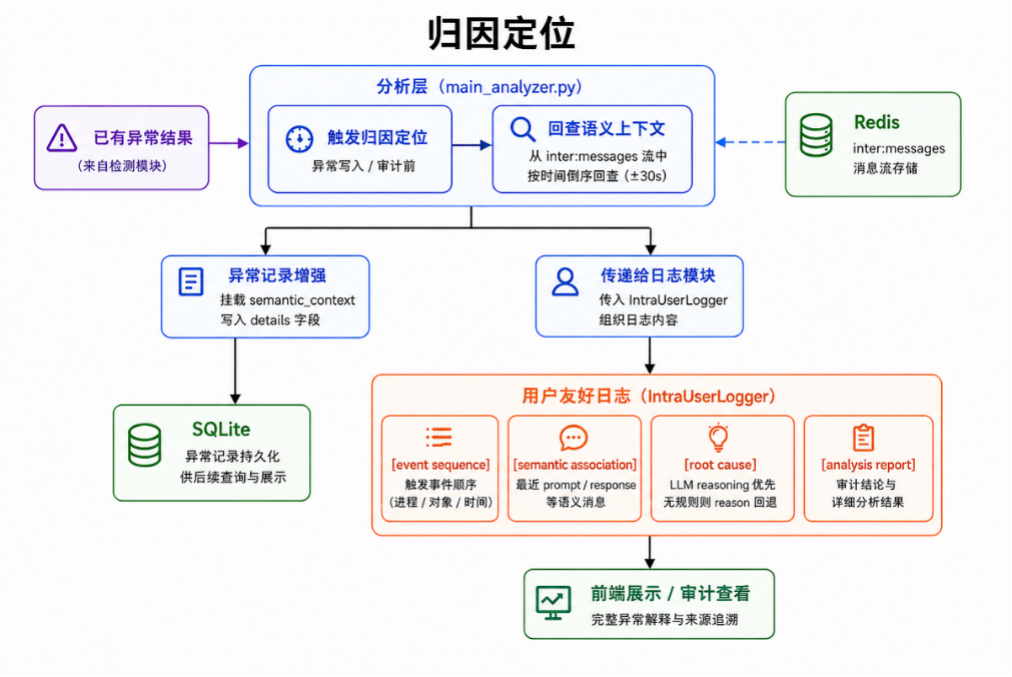
```
def detect(self,x,edge_index,edge_attr=None,topology=None):
    if not self._loaded:
        return {"anomaly_scores": np.zeros(x.shape[0]), "anomaly_nodes": [], ...}
    self._model.eval()
    self._classifier.eval()
    with torch.no_grad():
        anom_scores, feat_err, struct_err, _, z = \
                    self._model.compute_anomaly_scores(x_t, ei, topology=topology)
        cls_probs = torch.sigmoid(self._classifier(z).cpu()).numpy()
    final_scores = anom_scores * 0.5 + cls_probs * 0.5
    return {"anomaly_scores": final_scores,
            "anomaly_nodes": [i for i, s in enumerate(final_scores) if s > 0.5],
            "recon_error": float(feat_err), "cls_probability": cls_probs.tolist()}
```

代码片段 6：MonitorAgent 多模块加权融合。该片段展示协调器将 5 个检测器的风险分数按配置权重线性组合，并用保底提升的融合策略。

```
def _combine_scores(self,rule_result,gnn_result,temporal_result,message_result, agent_ids, semantic_rule_result=None):
    rule_scores = rule_result.get("scores", {})
    gnn_scores = gnn_result.get("anomaly_scores", [])
    temporal_scores = temporal_result.get("temporal_scores", {})
        sem_scores = semantic_rule_result. get( "agents", { } ) if semantic_rule_result else {}
    w_r, w_g, w_t, w_s = 0.35, 0.35, 0.15, 0.15
    scores = {}
    for i, aid in enumerate(agent_ids):
        r, g, t, s = rule_scores.get(aid, 0.0), float(gnn_scores[i]) if i < len(gnn_scores) else 0.0, \
                    temporal_scores.get(aid, 0.0), sem_scores.get(aid, 0.0)
        scores[aid] = min(max(w_r * r + w_g * g + w_t * t + w_s * s, max(r, g, t, s) * 0.8), 1.0)
    for anom in message_result.get("anomalies", []):
        src, dst, risk = anom. get( "src", "" ), anom. get( "dst", "" ), float(anom.get("risk", 0.5))
        if src in scores: scores[src] = min(max(scores[src], risk), 1.0)
        if dst in scores: scores[dst] = min(max(scores[dst], risk * 0.8), 1.0)
    return scores
```

4.4 归因定位

4.4.1 功能描述



归因定位用于在异常已经被检测出来之后，解决追溯异常来源的问题。不需要重新做一遍异常检测，而是在已有异常结果的基础上，回查异常发生前后的运行上下文，把相关的语义消息、触发对象、进程信息和审计结论串起来，形成更完整的来源解释。

这项能力主要承担三部分作用：补充异常发生前后的语义上下文，例如最近一次 prompt、response 或交互片段；关联异常对象和对应智能体、进程、时间顺序，避免只看到一条孤立告警；进一步组织上下文形成用户友好的 root cause 描述，便于日志展示、前端展示和后续审计说明。

当前实现中，归因定位能力由 `analysis` 层统一接入。`main_analyzer.py` 负责在异常写入和审计处理前回查语义上下文，并将结果挂载到异常详情或传递给日志输出模块；具体的根因解释则在`intra-agent`与 `inter-agent`两条链路中分别组织。

4.4.2 实现细节

系统在分析层维护 Redis 连接，并订阅运行时事件流，当某条异常需要落库或进入审计链路时，主分析器会调用`_enrich_semantic_context()`，从`inter:messages`流中按时间倒序回查最近的语义消息。回查时会限定智能体

ID，并设置 30 秒时间窗口，只提取异常发生前后最相关的 prompt / response 片段，从而避免把无关历史混入结果。

拿到语义上下文后，系统会把这些内容作为`semantic_context`写入异常记录的`details`字段，同时在单体审计链路中继续传给`IntraUserLogger`。这样一来，同一条异常既可以保存在 SQLite 中，便于后续查询和前端读取，也可以直接在用户友好日志中展示出更完整的事件前因后果。

在日志组织上，`intra_agent/user_log.py`会把异常记录拆成`[event sequence]`、`[semantic association]`、`[root cause]` 和 `[analysis report]`四个部分。前两部分负责展示触发顺序和最近语义消息，`[root cause]` 优先引用 LLM 审计返回的`reasoning`，如果没有命中 LLM 结果，则回退到规则层给出的`reason`。这种处理方式的好处是：简单规则异常可以快速给出解释，复杂异常又能接住后续语义审计的结论。

整体来看，归因定位并不直接参与规则命中，而是在异常已经产生之后，补充一层“上下文解释”，将运行时事件、消息流、审计推理和日志输出串联为一条更完整的说明链路。

4.4.3 核心代码片段

代码片段 1：语义上下文回查。该片段展示主分析器如何从 Redis 消息流中回查异常发生前的最近 prompt / response，并整理成 semantic_context 结构。

```
def _enrich_semantic_context(self, agent_id, pid, ts_ns):
        """根因识别：从 inter: messages 反向查找异常发生前的最近 prompt/response"""
    if not self.redis_client or not agent_id:
        return None
    lookback_window = 30 * 1_000_000_000  # 30s 回查窗口
    cutoff = max(0, ts_ns - lookback_window)
    try:
        raw = self.redis_client.xrevrange(
            MESSAGE_STREAM, "+", "-", count=20,
        )
        if not raw:
            return None
        messages = []
        for _stream_id, fields in raw:
            f = {self._safe_decode(k): self._safe_decode(v) for k, v in (fields or {}).items()}
            msg_ts = self._coerce_int(f.get("timestamp_ns", 0))
            msg_agent = f.get("agent_id", f.get("src", ""))
            msg_type = f.get("type", "")
            if msg_agent == agent_id and msg_ts >= cutoff and msg_type in ("prompt", "response"):
                data_preview = f.get("data", "")[:200]
                messages.append({
                    "timestamp_ns": msg_ts,
                    "type": msg_type,
                    "preview": data_preview,
                    "pid": self._coerce_int(f.get("pid", 0)),
                })
```

代码片段 2：异常详情挂载。该片段展示主分析器如何在异常写入前把回查到的语义上下文挂载到 details 中，供数据库、日志和前端统一使用。

```
def _push_anomaly(self, payload):
    self.anomaly_seq += 1
        details = dict( payload. get( "details", { } ) ) if isinstance(payload.get("details"), dict) else {}
    sem_context = self._enrich_semantic_context(
        agent_id=payload.get("agent_id", ""),
        pid=payload.get("pid", 0),
        ts_ns=payload.get("timestamp_ns", time.time_ns()),
    )
    if sem_context:
        details["semantic_context"] = sem_context

    record = {
        "anomaly_id": f"anom_{self.anomaly_seq:06d}",
        "rule": str(payload.get("rule", "UNKNOWN")),
        "severity": str(payload.get("severity", "LOW")),
        "timestamp_ns": int(payload.get("timestamp_ns", time.time_ns())),
        "pid": int(payload.get("pid", 0)),
        "tid": int(payload.get("tid", payload.get("pid", 0))),
        "comm": str(payload.get("comm", "unknown")),
                "agent_id": str( payload. get( "agent_id", self._agent_id_for_pid(payload.get("pid", 0)))),
        "object": str(payload.get("object", "-")),
        "reason": str(payload.get("reason", payload.get("description", "-"))),
        "details": json.dumps(details, ensure_ascii=False),
    }
    self.sqlite_store.persist_anomaly(record)
```

代码片段 3：根因日志组织。该片段展示 intra-agent 日志模块如何把事件序列、语义关联和 root cause 组合成可直接展示的日志块。

```
def _format_block(self, req, result, semantic_context):
    llm_review = result.get("llm_review") or {}
    event_lines = self._build_event_sequence(req, semantic_context)
    semantic_lines = self._build_semantic_section(semantic_context)
    root_cause = self._build_root_cause(req, result, llm_review)
    summary = self._build_summary(result, llm_review)
    ts_ns = int(req.get("timestamp_ns", 0) or 0)
    ts_text = self._format_ts(ts_ns)
    risk = llm_review.get("risk_level", req.get("severity", "LOW"))
    agent_id = req.get("agent_id") or f"agent_{int(req.get('pid', 0) or 0)}"
    object_text = req.get("object", "-")
    lines = [
        "=" * 54,
        f"[solo] {req.get('rule', 'UNKNOWN')} ({risk})",
            f"time={ts_text}  pid={req.get('pid', 0)}  tid={req.get('tid',
req.get('pid', 0))}",
              f"agent={agent_id}  comm={req.get('comm', 'unknown')}
object={object_text}",
        "-" * 54,
        "[event sequence]",
        *event_lines,
        "[semantic association]",
        *semantic_lines,
        "[root cause]",
        f"  {root_cause}",
        "[analysis report]",
        f"  {summary}",
    ]
    return "\n".join(lines)
```

4.5 CoreSight-showcase

4.5.1 功能描述



CoreSight-showcase 是本项目的前端应用层，主要用于异常结果展示、链路呈现和系统运行状态查看。将后端已完成的异常链路、审计结果和整体运行情况整理成一个更容易理解的交互界面，相比直接查看控制台日志或者数据库表，这个前端更强调把“过程”展示出来，让用户能看懂系统发现问题的整体流程细节。

结合当前实现，前端的主要功能可以概括为三部分：第一部分是对话式入口，用户可以直接通过自然语言触发展示流程；第二部分是模式化展示，前端目前支持 single、interactive`和 all 三种模式，分别对应单点审计链路、交互式多智能体异常链路以及联合展示链路；第三部分是链路可视化，系统会把分析过程组织成分步骤的流式输出，便于用户查看系统处理过程。

4.5.2 实现细节

前端项目位于`ebpf-monitor-showcase`目录下，采用 Next.js 15 和 React 19 实现，页面入口是`app/page.tsx`，主界面由`app/assistant.tsx`加载。对话运行时基于`AssistantRuntimeProvider`、`useChatRuntime`和`AssistantChatTransport`构

建，用户输入会统一发送到`app/api/forward-chat/route.ts`，后端再根据关键词判断当前是`single`、`interactive`、`all`还是普通问答模式。

在界面组织上，首页保留了对话式展示外壳，但内容已经按当前展示需求做了调整。欢迎区域里预置了模式入口和示例问题，用户可以直接触发单点、交互或全量展示链路。消息区也不再只是普通文本回复，而是通过`components/assistant-ui/thread.tsx`和`components/assistant-ui/flow-message.tsx`把结果解析成分步骤的流程节点，用来展示从`showcase`、`intra-agent`、`inter-agent`到`audit`的推进过程，实时展示链路进度。

在数据来源上，前端读取真实运行结果，而不是只放静态示例。`app/api/forward-chat/route.ts` 会优先读取监控库里的异常记录、审计结果和`intra-agent` 日志；`lib/server/monitorApi.ts` 负责从主项目生成的 SQLite 数据中读取 `events`、`anomalies`、`audit_results` 等内容；`app/api/overview`、`app/api/events`、`app/api/anomalies`、`app/api/audit-results` 等接口则负责把这些数据整理成前端可以直接消费的格式。另外，`components/showcase-console.tsx` 还提供了总览控制台页面，用来集中展示总体统计、最新异常和最新审计结果。

5 代码说明

5.1 项目目录结构

5.1.1 整体

```
ebpf-multi-agent-monitor/
│
├── Makefile                    # 顶层构建编排
├── README.md                   # 项目概述
├── requirements.txt            # Python 依赖列表
│
├── build/    # 构建产物
│   ├── sys_behavior.bpf.o      # 编译后的 BPF 目标文件
│   ├── sys_behavior.skel.h     # BPF 骨架头文件
│   ├── ssl_sniffer.bpf.o       # 编译后的 BPF 目标文件
│   ├── ssl_sniffer.skel.h      # BPF 骨架头文件
│   ├── loader                  # 启动程序
│   └── ringbuf_handler         # 事件捕获与数据处理程序
│
├── docs/     # 设计文档
│   ├── llm_audit_design.md     # LLM 审计系统设计
│   ├── monitor_runtime_design.md # 运行链路说明
│   └── Redis 数据接口规范.md    # Redis 数据接口规范
│
├── logs/     # 运行时日志与持久化
│   ├── inter_log/              # 智能体间分析日志
│   │   └── anomaly_*.log
│   └── intra_log/              # 单点分析日志
│       └── analysis.db         # 事件/异常/审计结果
│
├── test/*    # 测试程序
│
├── .vscode/
│   └── c_cpp_properties.json   # C/C++ IntelliSense 配置
│
└── src/ # 主源代码
    │
    ├── bpf/ # eBPF 内核探针
    │   ├── vmlinux.h           # 自动生成的内核 BTF 头文件
    │   ├── common.h            # 共享结构体/枚举/常量
    │   ├── filter.h            # Agent 识别过滤器
    │   ├── sys_behavior.h      # sys_event 结构体
    │   ├── sys_behavior.bpf.c  # 系统调用探针
    │   ├── sys_behavior.c      # sys_behavior 用户态加载器
```

```
│   ├── ssl_sniffer.h              # probe_SSL_data_t 结构体
│   ├── ssl_sniffer.bpf.c          # SSL/TLS 明文捕获
│   ├── ssl_sniffer.c              # ssl_sniffer 用户态加载器
│   ├── agent_registry.h           # 统一注册函数声明
│   └── agent_registry.c           # 统一 register_agent() 实现
│
├── config/*  # 配置文件
│
├── user/      # 用户态 C 程序
│   ├── Makefile                   # 构建规则
│   ├── loader.c                # 加载器
│   ├── ringbuf_handler.c          # Ring buffer 事件分发
│   ├── redis_client.h             # Redis 推送接口声明
│   ├── redis_client.c             # Redis 连接生命周期
│   ├── agent_tracker.h            # Agent 跟踪接口声明
│   ├── agent_tracker.c            # Agent 注册
│   ├── metrics_aggregator.h       # 指标聚合接口声明
│   ├── metrics_aggregator.c       # 时间窗口指标聚合
│   ├── topology_builder.h         # 拓扑构建接口声明
│   ├── topology_builder.c         # Agent 间边关系
│   ├── event_serializer.h         # 事件序列化接口声明
│   └── event_serializer.c         # JSON 编码、base64、路径工具函数
│
└── analysis/*                  # Python 分析引擎
```

5.1.2 intra_agent

```
ebpf-multi-agent-monitor/
├── src/analysis/
│   ├── main_analyzer.py              # 主分析器，负责事件分发、调用单体检测模块并衔接后续处
理流程
│   ├── audit_engine.py               # 审计复核模块，对单体检测结果进行进一步判断
│   ├── sqlite_store.py            # 结果落库模块，用于持久化异常与审计结果
│   └── intra_agent/
│       ├── main.py                   # 单体检测核心实现，包含 IntraAgentAnalyzer 及主要规则
检测逻辑
│       └── user_log.py            # 单体日志输出模块，负责生成用户友好日志
├── src/config/
│   ├── app_config.yaml               # 工作区、LLM、运行参数等全局配置
│   └── rules.yaml                    # 单体审计规则与黑白灰名单配置
└── logs/intra_log/                   # 单体检测链路的日志输出目录
```

5.1.3 inter_agent

```
inter_agent/
├── __init__.py                       # 包入口
├── conftest.py                       # pytest 共享 fixtures
```

```
├── main.py                          # 入口：--once / --simulate / --live
├── requirements.txt                 # 依赖列表
│
├── autogen_captured/                # AutoGen 原始捕获数据（.jsonl）
│   ├── autogen_hierarchical_*.jsonl （×4，层级拓扑）
│   └── autogen_peer-to-peer_*.jsonl （×4，点对点拓扑）
│
├── config/                          # 配置加载
│   ├── __init__.py
│   ├── default.yaml                 # 默认配置（检测阈值、路径、LLM 开关）
│   └── loader.py                  # YAML → dict 加载器
│
├── data/                            # 数据采集、生成与注入
│   ├── __init__.py
│   ├── anomaly_injectors.py         # 19 种异常类型注入器注册表
│   ├── autogen_scenarios.py         # AutoGen 语义场景注入（10 种）
│   ├── baseline_calibrator.py       # 基线阈值校准
│   ├── collector.py                 # RedisCollector（从 eBPF 实时读取）
│   ├── data_recorder.py             # 观测数据记录器
│   ├── dataset_builder.py           # 训练/验证/测试集构建
│   ├── features.py                  # 节点/边特征维度定义与提取
│   ├── schema.py                    # 数据字段 schema 定义
│   ├── simulator.py                 # 模拟器（生成正常+注入异常）
│   ├── stateful_generator.py        # 时序序列生成器（含状态）
│   ├── text_encoder.py              # 文本编码（LLM embedding）
│   └── vm_autogen_collector.py      # VM 端 AutoGen 采集函数
│
├── detection/                       # 5 大检测器
│   ├── __init__.py
│   ├── gnn_detector.py              # GNN 图神经网络检测
│   ├── message_detector.py          # 消息级异常检测（风暴、洪泛等）
│   ├── propagation_analyzer.py      # 传播路径分析
│   ├── semantic_rule_checker.py     # 语义规则检测（15 种类型）
│   └── temporal_detector.py         # 时序检测器
│
├── examples/                        # 示例
│   └── train.py                     # GNN 模型训练流水线
│
├── graph/                           # 图构建
│   ├── __init__.py
│   ├── builder.py                   # AgentInteractionGraph（交互图构建）
│   └── utterance_graph.py           # 话语/语义图
│
├── intervention/                    # 干预响应
```

```
│   ├── __init__.py
│   └── topology.py                 # 拓扑干预（隔离/降级）
│
├── models/                         # 模型文件
│   └── anomaly_detector.pth        # GNN 训练权重
│
├── monitor/                        # 核心监控代理
│   ├── __init__.py
│   └── agent.py                    # MonitorAgent（主控制器，6 检测器编排）
│
├── scripts/                        # 可执行脚本
│   ├── convert_jsonl_to_split.py       # .jsonl 原始数据 → train/val/test 拆分
│   ├── demo_all_anomalies.py       # 19 种异常类型全覆盖演示
│   ├── demo_large_scale.py         # 大规模 agent 场景演示
│   ├── eval_combined_system.py     # 组合系统评估
│   ├── eval_full_system.py         # 全系统评估
│   ├── run_1_monitor.sh            # 启动 eBPF loader + ringbuf
│   ├── run_2_demo_types.sh         # 运行全部 19 异常类型演示
│   ├── run_3_demo_scale.sh         # 运行大规模演示
│   └── vm_runtime_collector.py     # VM 运行时采集+检测流水线
│
├── tests/                          # 单元/集成测试
│   ├── __init__.py
│   ├── test_detection.py           # 检测器测试
│   ├── test_integration.py         # 集成测试
│   ├── test_schema.py              # schema 验证
│   └── test_semantic_rules.py      # 语义规则评估
│
├── training_data/                  # 训练/验证/测试数据集
│   ├── meta.json                   # 元数据：2400 obs，1202 normal + 1198 anomaly
│   ├── test_labeled.json           # 测试集（359 obs，带 label + _anomaly_type）
│   ├── train_labeled.json          # 有监督训练集（1682 obs）
│   ├── train_unlabeled.json        # 无监督训练集（842 obs，仅 normal）
│   └── val_labeled.json            # 验证集（359 obs）
│
└── utils/                          # 工具
    ├── __init__.py
    └── logger.py                   # 日志格式化工具
```

5.2 关键类/函数说明

类/函数名	所在文件	功能说明	输入	输出

process_data_exit	ssl_sniffer.bpf.c	SSL 明文数据统一装配	(void *buf, int len, int rw, bool is_handshake, enum sniff_engine engine)	int(0)，通过 ringbuf 输出 probe_SSL_data_t
probe_SSL_rw_enter	ssl_sniffer.bpf.c	SSL_read/SSL_write 函数入口 uprobe	(void *ssl, void *buf, int num)	int(0)
probe_SSL_read_exit / probe_SSL_write_exit	ssl_sniffer.bpf.c	SSL_read/SSL_write 返回 uretprobe	ctx	int(0)，通过 process_data_exit 输出
go_tls_write_enter	ssl_sniffer.bpf.c	Go Conn.Write 入口 uprobe	ctx	int(0)
go_tls_read_exit	ssl_sniffer.bpf.c	Go Conn.Read 返回 uretprobe	ctx	int(0)
submit_raw_event	sys_behavior.bpf.c	纯文本协议数据统一装配	(__u32 pid, __u32 fd, const char *buf, size_t count, int direction, enum raw_data_source source, __u32 peer)	void，通过 ringbuf 输出 raw_data_event
submit_sys_event	sys_behavior.bpf.c	系统事件统一装配	(__u32 event_type, __u64 pid_tgid, const char *filename)	void，通过 ringbuf 输出 sys_event
submit_connect_event	sys_behavior.bpf.c	Connect 事件装配与智能体对端识别	(__u64 pid_tgid, struct sockaddr	void

			*uservaddr, int addrlen)	
submit_msghdr_event	sys_behavior.bpf.c	sendmsg/recvmsg 数据装配	(int fd, struct user_msghdr *msg, int direction)	void
read_msghdr_payload	sys_behavior.bpf.c	提取 iovec 载荷	(struct user_msghdr *msg, __u8 *dst, __u32 max_len)	int 实际拷贝字节数
trace_sched_process_fork	sys_behavior.bpf.c	fork tracepoint	(struct trace_event_raw_sched_process_fork *ctx)	int(0)，写入 monitored_pids 和 ringbuf
promote_to_sub_agent	sys_behavior.bpf.c	子智能体识别	(__u32 pid)	bool
has_agent_protocol_fingerprint	sys_behavior.bpf.c	智能体通信协议识别	(const __u8 *buf, __u32 len)	bool
resolve_listen_peer	sys_behavior.bpf.c	查 listen_map 判断 connect 身份	(__u16 family, __u32 saddr, __u16 sport, __u32 *peer_agent_id, __u32 *peer_pid, char *peer_name)	int
attach_probes_dynamically	ssl_sniffer.c	uprobe 挂载	(struct ssl_sniffer_bpf *skel, int *pids, int count)	void
find_boringssl_offsets	ssl_sniffer.c	BoringSSL 字节码定	(const char *binary_path	struct boringssl_off

		位	)	sets(found+ssl_write+ssl_read+ssl_do_handshake)
handle_ssl_event	ringbuf_handler.c	SSL ringbuf 回调	(void *ctx, void *data, size_t len)	int 0
handle_raw_event	ringbuf_handler.c	纯文本 ringbuf 回调	(void *ctx, void *data, size_t len)	int 0
handle_sys_event	ringbuf_handler.c	系统事件 ringbuf 回调	(void *ctx, void *data, size_t len)	int 0
IntraAgentAnalyzer	src/analysis/intra_agent/main.py	单智能体异常检测核心类，维护每个 PID 的行为状态、滑动窗口统计和规则检测逻辑	运行时事件 event_data	待审计请求列表 reqs
IntraAgentAnalyzer._get_stats	src/analysis/intra_agent/main.py	按 PID 获取或初始化单体行为统计，并在窗口超时后重置计数器	pid、ts_ns、comm	当前 PID 对应的统计状态 stats
IntraAgentAnalyzer.analyze_event	src/analysis/intra_agent/main.py	根据事件类型执行规则检测，识别 shell 执行、敏感文件访问、越界删除、异常网络连接等行为	标准化事件 event_data	命中的待审计请求列表 reqs
IntraAgentAnalyzer._maybe_loop_req	src/analysis/intra_agent/main.py	根据窗口内文件类操作、网络类操作和对象集中度判断是否存在逻辑循环风险	event_data、当前统计状态 stats	逻辑循环待审计请求或 None
AuditEngine	src/analysis/audit_engine.py	单体审计核心类，负责加载规则配置、黑白灰名单分流和 LLM 复核	待审计请求 req	审计结果 result

MessageAnomalyDetector.detect	message_detector.py	检测 4 种消息异常模式	obs: dict	{"anomalies", "max_risk"}
PropagationPathAnalyzer.analyze	propagation_analyzer.py	BFS 异常传播路径分析	anomalous_nodes, anomalous_edges	{"propagation_paths", "source_agents", "total_risk"}
TemporalAnomalyDetector.detect	temporal_detector.py	z-score 时序异常计算	-（使用内部_history）	{"temporal_scores", "anomaly_agents"}
RuleBasedDetector.detect	gnn_detector.py	阈值规则检测	agents_data: dict	{"scores", "anomalies"}
AnomalyDetector.detect	gnn_detector.py	DOMINANT 图异常检测	x, edge_index, edge_attr, topology	{"anomaly_scores", "anomaly_nodes", "recon_error"}
AgentInteractionGraph.to_pyg_data	builder.py	构建 PyG 格式图数据	-（使用内部_graph）	(x, edge_index, edge_attr)

6 实验说明

6.1 实验环境

项目	版本/配置
操作系统	6.17.0-35-generic
编程语言	c 、python 、html
核心依赖	libbpf、Redis、PyYAML、NumPy、NetworkX、Next.js 15.5.2、React 19、Tailwind CSS 4
硬件环境	tel Core i7-13700H，16 vCPU，16 GB 内存

6.2 依赖安装

```
# 1. 安装系统依赖
sudo apt update
sudo apt install -y clang llvm make cmake libelf-dev zlib1g-dev libbpf-dev bpftool redis-server libhiredis-dev python3-pip
# 2. 安装主监控项目 Python 依赖
cd ./ebpf-multi-agent-monitor
pip install -r requirements.txt
# 3. 安装 inter-agent 依赖
cd ./ebpf-multi-agent-monitor/src/analysis/inter_agent
pip install -r requirements.txt
# 4. 编译 eBPF 用户态与内核态程序
cd ./ebpf-multi-agent-monitor/src/user
make
# 5. 安装前端展示系统依赖
cd ./ebpf-monitor-showcase
npm install
```

6.3 编译/构建说明

```
cd ./ebpf-multi-agent-monitor
make clean && make
```

6.4 使用说明

6.4.1 快速开始

```
# 1. 启动 Redis
redis-server --daemonize yes
# 2. 启动 eBPF 采集端
cd ./ebpf-multi-agent-monitor
sudo build/loader
sudo build/ringbuf_handler
# 3. 启动单点分析模式
python3 src/analysis/main_analyzer.py --mode single
# 4. 启动交互分析模式
python3 src/analysis/main_analyzer.py --mode interactive
# 5. 启动全量分析模式
python3 src/analysis/main_analyzer.py --mode all
# 6. 运行 inter-agent 监测脚本
cd ./ebpf-multi-agent-monitor/src/analysis/inter_agent/scripts
bash run_1_monitor.sh
bash run_2_demo_types.sh
bash run_3_demo_scale.sh
# 7. 启动前端展示系统
cd ./ebpf-monitor-showcase
npm run dev
```

6.4.2 功能演示

系统支持单点、交互和全量三种运行方式，对应不同的分析场景。

single 模式：用于展示单个智能体的异常检测与审计流程，重点观察异常命令执行、敏感文件访问、工作区越界、逻辑循环以及语义与控制异常等结果，相关输出可在终端、SQLite 数据库和 logs/intra_agent/日志中查看。

interactive 模式：用于展示多智能体之间的交互异常分析过程，可观察消息传播、交互拓扑和关联异常识别情况，结果可结合后端日志与前端链路界面查看。

all 模式：用于同时展示单点异常检测、交互异常检测和审计输出，能够看到更完整的整体链路。

6.4.3 参数说明

参数	类型	默认值	说明
--help	布尔	false	显示帮助信息
--agent	字符串	监测全部智能体	是否指定监测智能体对象
PID	整数	监测全部进程	是否指定监测进程对象
--mode	字符串	all	选择分析模式，可选 single、interactive、all
--once	布尔	false	inter-agent 单次检测模式
--simulate	布尔	false	inter-agent 模拟数据运行模式
--live	布尔	false	inter-agent 实时 Redis 轮询模式
--stored	布尔	false	使用本地存储数据代替实时 Redis
--window	整数	30	轮询间隔（s）
--rounds	布尔	200	演示轮数
--topology	字符串	"hierarchical"	多智能体协作拓扑："hierarchical"/"peer-to-peer"
DEEPSEEK_API	环境变量	空	LLM 审计接口密钥，用于灰名单

_KEY			与复杂异常复核
loop_window_ns	整数	30s	intra-agent 滑动统计窗口，用于统计文件、网络和标准输入输出行为
event_alert_cooldown_ns	整数	5s	同类事件告警冷却时间，避免短时间重复刷屏
loop_alert_cooldown_ns	整数	30s	逻辑循环告警冷却时间
max_file_samples	整数	8	单个 Agent 窗口内保留的文件访问样本数量
llm.model	配置项	deepseek-chat	LLM 审计使用的模型名称

7 系统测试

7.1 功能测试

测试功能	测试方案	测试结果	测试说明
智能体启动识别	真实启动测试	Pass	对 Claude、OpenCode 等智能体正确识别，对用户指定智能体正确识别
智能体行为捕获	多场景模拟数据注入测试	Pass	对 fork, execve, open, unlink, chmod, bind, connect, close, stdin read, stdout/stderr write, sendto, recvfrom, sendmsg, recvmsg, sched_switch, SSL_write, SSL_read, SSL_write_ex, SSL_read_ex, go_tls_write, go_tls_read, SSL_do_handshake 事件进行测试
智能体明文捕获	多类型智能体真实通信验证	Pass	测试 Claude、Gemini CLI 等多类型单智能体
智能体资源占用捕获	模拟数据测试	Pass	模拟智能体启动场景测试数据库中抓取到的四个级别资源占用信息
intra-agent 单体规则检测	单点模式真实/构造事件测试	Pass	在 single 模式下触发 execve、open、unlink、connect 等事件，检查 IntraAgentAnalyzer.analyze_event 是否生成待审计请求
LLM 审计复核	黑名单异常请求测试	Pass	对黑名单异常触发 AuditEngine.llm_review，检查返回结果是否包含 decision、risk_level、reasoning、evidence 等字段
黑白灰名单分流	规则配置与样例请求测试	Pass	分别构造白名单、灰名单、黑名单请求，检查 AuditEngine.classify_intra_req 是否返回 WHITE、GRAY、

			BLACK
SQLite 落库完整性	真实实时数据测试	Pass	检查原始事件、异常记录和审计结果是否写入 analysis.db，并可被前端接口读取
多智能体规则检测器正常判定	真实监测数据测试	Pass	捕获大量真实无异常数据进行检测，所有 agent 风险分数均为 0.0，无异常标记
多智能体规则检测器异常捕获	模拟数据测试	Pass	对正常数据进行异常注入，测试所有规则检测器可检测的异常
消息检测器正常判定	真实检测数据测试	Pass	正常观测无消息异常
消息检测器内部告警	模拟数据测试	Pass	对正常数据进行异常注入，检查是否准确对异常内容告警
全流水线正常处理	真实实时数据测试	Pass	通过 3 个启动脚本定位流水线异常模块
日志报告结构完整性	真实实时数据测试	Pass	检查 log 中时间戳、风险、异常节点、状态视图是否均存在

7.2 性能测试

测试功能	测试方案	测试结果	测试说明
数据捕获开销	大量模拟数据与真实数据混合测试	额外开销 2.72%	全部探针加载产生额外指令开销平均数值为 2.72%
数据抓取率	模拟真实数据混合测试	100%	事件捕获率 100%
智能体间异常识别准确率（测试集）	真实数据测试集	Precision=0.7788 Recall=0.8212 F1=0.7903 AUC=0.8506	Test: 359 obs

智能体间异常识别准确率（实时）	真实实时数据+异常注入	Precision=0.8261 Recall=1.0000 F1=0.9408 AUC=0.9600	TP (injected+detected)=19 FN (injected+missed)=0 FP (normal+flagged)=4 TN (normal+passed)=77
智能体间异常响应时间	真实实时数据	13ms/round	1.3s/100 rounds

8 研发过程记录

8.1 遇到的问题及解决方法

8.1.1 问题一

现象描述：loader 启动 SSL 引擎时，readlink("/proc/PID/exe")返回的路径末尾带有(deleted)后缀，后续所有 uprobe 挂载均因 open()返回 ENOENT 而失败，SSL 探针挂载全部失败。

根因分析：观察到路径末尾带有(deleted)后缀，(deleted) 来源于 Agent 进程启动后，其二进制文件被 npm 更新覆盖，内核不会回收旧 inode 但 dentry 已从目录树移除。fs/proc/base.c 中 proc_exe_link()检测到 d_unlinked()后，在 readlink 结果后自动追加(deleted)。代码将 readlink 返回的含(deleted)完整字符串直接作为文件路径传入 ，调用 open()时内核按字面路径查找文件，导致返回 ENOENT，从而引起后续失败。

解决方法：在 Linux 中，运行中进程的 inode 不会被销毁。直接把 Linux VFS 的 /proc/<pid>/exe 作为文件路径传入，即在 readlink 后增加(deleted)检测，若命中则回退到/proc/<pid>/exe 路径。

相关代码/提交：commit 0330f02c28e40e5174f7b9d4776944eb016d5337

Author: Charlie <3481963967@qq.com>

Date: Mon Jun 8 18:10:47 2026 +0800

fix multi:增量检查运行的多个智能体

```
1    snprintf(exe_path, sizeof(exe_path), "/proc/%d/exe", target_pid);
2    if (strstr(real_exe, " (deleted)") != NULL){
3        target_bin_path = exe_path;
4    }
```

8.1.2 问题二

现象描述：在语义异常检测模块的早期开发中，尝试使用 LLM 直接对 agent 间交互拓扑和消息内容进行异常判定，将观测数据中的 agent 指标、交互边列

表、消息序列拼接为结构化提示词，要求模型判断是否存在异常并给出风险分数。实验发现以下问题：LLM 对复杂拓扑结构的异常（如 group_split 组分裂、supervisor_bypass 越级通信）无法准确识别，经常将正常的交互模式误判为异常；模型倾向于给出偏高的风险分数（通常 ≥ 0.7），即使输入 SAFE_METRICS 基线数据也产生大量误报；输出不稳定，同一观测多次调用 API 可能得到不同的判定结果，无法保证检测的可复现性。

根因分析：根本原因有三方面：拓扑感知能力不足，LLM 以文本序列理解图结构，当 agent 数量增至 6 个、交互边超过 20 条时，模型难以在提示词上下文中准确维护完整的邻接关系，更无法执行 BFS 级别的路径分析来区分正常交互与 group_split 式的组间低权重分裂；阈值校准缺失，LLM 的输出概率分布未经领域数据校准，缺乏类似 fs_unlink_count >= 2 的精确数值边界，导致在正常值附近与异常值之间无法做出一致判定；成本与延迟不可控，每次检测需调用远程 API，单次延迟在 500ms-3s 之间，而系统预期检测延迟在毫秒级，LLM 方案的总耗时远超可接受范围。

解决方法：最终采用规则注册表模式替代 LLM 方案，针对问题提出以下改进方法：设定确定性规则引擎，将异常类型编码为纯结构化检查函数，每条规则包含精确的数值阈值；数据驱动阈值校准，所有阈值基于大量正常观测的指标分布进行校准，确保正常数据下所有规则的风险分数为 0.0，仅在指标超出正常分布范围时触发；将规则检测改为纯本地计算，单条规则复杂度 O(1)，规则全量执行耗时<1ms，且同一观测始终产生一致的检测结果。同时通过字典为每条规则配对了证据字段提取函数，可在检测到异常时精准返回触发原因，提升可解释性。

相关代码/提交：commit 1104d27b82004d8171f8bfac385c28b8a5ad6ce2

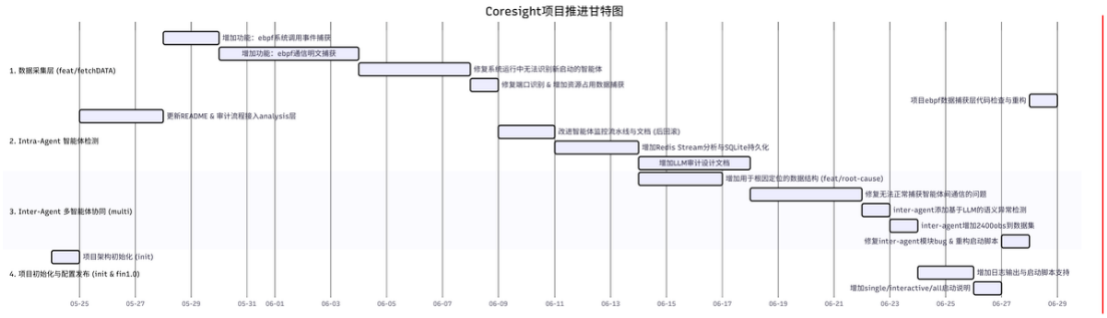
Author: 熔岩芝士烤冷面

Date: Jun 24,2026 7:57pm GMT+0800

feat multi: replace LLM semantic detector with rule-based checker

8.2 设计迭代记录

迭代版本	变更内容	变更原因
init	初始化项目框架	初始化
feat/fetchDATA	增加功能捕获数据	新增功能
feat/simulated-data-inter-agent	增加功能智能体异常识别	新增功能
feat/root-cause-analysis	增加功能根因定位	新增功能
multi	支持多智能体协作	扩展
feat/semantic-anomaly-detection	增加功能多智能体协作场景下异常识别与根因定位	新增功能
feat/fin1.0	增加日志	新增功能与代码审查



9 未来规划

9.1 待完成工作

序号	工作内容	优先级	预计完成时间
1	扩展对更多类型 Agent 的支持	高	2026. 7. 15
2	拆分多个 ring buffer 提升负载	中	2026. 7. 30
3	提升 GNN 对更多、更复杂异常的捕获能力	中	2026. 8. 15

9.2 优化方向

intra-agent 当前已经完成了单体行为检测、黑白灰名单审计、LLM 复核、日志输出和结果落库等基础链路。后续可以从以下几个方向继续优化：

1. 检测策略自适应优化：当前规则阈值主要依赖经验配置，例如逻辑循环检测中的时间窗口、文件样本数量和高频操作阈值；黑白灰名单也主要基于规则名和路径标记进行分流。后续可以结合真实运行数据、任务阶段、调用来源和历史行为画像，对规则阈值与名单策略进行动态调整，降低误报和漏报。
2. LLM 审计与归因能力增强：当前 LLM 复核已经能够返回结构化的 decision、risk_level、reasoning 和 evidence，归因定位也能回查异常前后的 prompt/response。后续可以增加相似请求缓存、批量复核和本地兜底机制，并进一步融合进程树、文件访问序列、网络连接目标和多轮对话上下文，使 root cause 从单条语义消息扩展为更完整的行为链路。

inter-agent 的架构设计为检测器的水平扩展预留了接口，未来尝试从以下方向进一步优化：

1. 当前 DOMINANT 模型使用 GCN/GAT 编码器在 CPU 环境下推理，计划引入图采样（GraphSAINT/ClusterGCN）支持更大规模的 agent 拓扑。

2. SemanticRuleChecker 的阈值目前基于 val_labeled.json 静态校准，计划引入在线贝叶斯更新机制，使其在新场景下自适应调整触发边界。
3. PropagationPathAnalyzer 的 BFS 搜索目前以 agent ID 字符串匹配为代价，拓扑结构变化时可结合强化学习选择最优的 max_hops 参数。
4. 系统暂未集成多模态数据源（如网络包级特征），未来可融合 eBPF 更细粒度的 TCP_STATE 事件构建 agent 间通信指纹库。

10 开源/AI 说明

10.1第三方开源组件

组件名称	许可证	用途说明	是否修改
bpftool	GNU GPL v2.0-only and BSD 2-Clause	用于 bpftool gen skeleton 命令，将编译后的.bpf.o 文件转化为 C 头文件形式的 BPF skeleton，包含 map 定义、程序加载、attach 等操作的封装函数	否
libbpf	BSD 2-clause license and GNU LGPL v2.1 license	用于 bpf 程序验证与加载、挂载探针、map 操作、ring buffer 事件读取，利用 CO-RE 特性	否
hiredis	BSD license	用于向 Redis 推送捕获数据	否

10.2参考的开源项目/论文

项目名称/论文标题	作者/来源	参考内容	引用位置
bpf-developer-tutorial	https://g...	该项目为本团队提供了 eBPF 程序开发的基础教程	数据捕获中的 eBPF 探针挂载与 Ring Buffer 设计
Don't Let the Claw Grip Your Hand: A Security Analysis and Defense Framework for OpenClaw	https://a...	该论文以 OpenClaw 本地代码 Agent 为对象，分析 shell 命令、文件读写、编辑操作中的间接提示注入、沙箱逃逸、权限扩大和资源攻击风险，并设计 Allowlist、Semantic Judge、Pattern Matching、Sandbox Guard 等 HITL 防御流水线	intra-agent 中异常命令执行、敏感文件访问、工具行为审计以及黑白名单+ LLM 复核机制
AgentCgroup: Understanding and Controlling OS	https://a...	该论文系统分析 AI coding agent 在 OS 层面的工具调用、初始	intra-agent 中资源占用捕获、运行时行为统计、逻辑循环和资

Resources of AI Agents		化、CPU、内存和资源突增特征，并提出基于 eBPF 与 cgroup 的细粒度资源观测和控制方案	源空转检测
TrinityGuard	[2603.1...	该项目提供了风险分类框架	inter-agent 和 intra-agent 的异常类别参考
GUARDIAN	https://g...	该项目启发多智能体异常检测模块采用图拓扑建模和多层检测架构	inter-agent 中的 GNN 建模与训练
G-Safeguard	[2502.1...	该项目为多智能体异常检测模块拓展图建模带来一定启发	inter-agent 中利用话语图进行通信异常检测

10.3AI 辅助生成内容说明

内容类型	AI 工具	使用场景	人工审核情况
前期调研	TRAE	辅助进行前期调研、文章阅读、对比分析	针对调研内容人工进行查找复核，确定项目整体方向和架构
项目文档撰写	DeepSeek	项目设计方案文档的语句润色与格式规范化，对文档文字表述进行优化。	人工全文审阅，确认技术描述与实际实现一致，修正 AI 生成内容中与代码实现不符的细节描述。
代码注释与函数说明	GitHub Copilot	代码编写完成，AI 辅助生成函数文档字符串和行内注释，提高代码可读性。	核心代码逻辑手动编写调试，AI 辅助注释文本和代码审查，注释内容已核对。
测试脚本	OpenCode	辅助生成用于异常注入测试的测试脚本和性能测试的模拟数据。	测试脚本的核心注入逻辑由团队成员独立实现，AI 生成代码经过审查与改造，确保注入数据符合标准。

11 附录

附录 A：变更日志

日期	变更人	变更内容
2026.5.25	Charlie	项目架构初始化
2026.5.28	熔岩芝士烤冷面	更新 intra-agent README&将 intra-agent 审计流程接入 analysis 分析层
2026.5.30	Charlie	增加功能 ebpf 系统调用事件捕获
2026.6.1-4	Charlie	增加功能 ebpf 通信明文捕获
2026.6.8	Charlie	修复系统运行中无法识别新启动的智能体
2026.6.9	Charlie	修复端口识别&增加资源占用数据捕获
2026.6.11	熔岩芝士烤冷面	改进智能体监控流水线与文档&后续回滚该次改动
2026.6.14	熔岩芝士烤冷面	增加 Redis Stream 分析&SQLite 持久化能力
2026.6.17	Charlie	增加用于根因定位的数据结构
2026.6.18	熔岩芝士烤冷面	增加 LLM 审计设计文档
2026.6.22	熔岩芝士烤冷面	修复无法正常捕获智能体间通信的问题
2026.6.23	熔岩芝士烤冷面	inter-agent 添加基于 LLM 的语义异常检测模块
2026.6.24	熔岩芝士烤冷面	inter-agnet 增加 2400obs 到数据集
2026.6.26	熔岩芝士烤冷面	增加日志输出与启动脚本支持
2026.6.27	熔岩芝士烤冷面	增加 single/interactive/all 三种模式启动命令说明
2026.6.28	熔岩芝士烤冷	修复 inter-agent 模块 bug&代码检查与优化&重构启动脚

	面	本
2026.6.29	Charlie	项目 ebpf 数据捕获层代码检查与重构